

SAS — Shipment Alerting System

Architecture & Data-Flow Guide

Dart Global Logistics

How the frontend, APIs, database and syncing fit together — with the exact files for each piece. Focused on the milestone, alerting and field-registry layer.

Stack: Next.js 15 (App Router) • Python Flask • Supabase / PostgreSQL • APScheduler

1. The big picture

SAS has three layers that talk over HTTP and the Supabase client:

- **Frontend** — a Next.js app (folder SAS/). Components render pages and call the backend over HTTP.
- **Backend** — a Flask API (folder Backend/) on `http://127.0.0.1:5000`. All logic and DB access lives here.
- **Database** — Supabase (hosted PostgreSQL). Both layers connect, but the backend does the heavy lifting.

```
Browser (Next.js pages/components)
| fetch('http://127.0.0.1:5000/api/...')
v
Flask API (Backend/app.py -> routes/*.py -> services/*.py)
| supabase client (service key)
v
Supabase / PostgreSQL <-- CargoWise sync (scheduled) writes shipments
```

Two ways the frontend reaches data: (1) **through the Flask API** (the normal path), and (2) **directly to Supabase** with the anon key for a few simple reads. The API path is the one that matters for your work.

2. How the frontend connects to the API

Components call the Flask base URL, read from an env var with a localhost fallback:

```
// SAS/lib/services/shipment.service.ts
const FLASK_API = process.env.NEXT_PUBLIC_API_URL ?? 'http://127.0.0.1:5000'
fetch(`${FLASK_API}/api/shipments`)
```

- **Service layer** — `SAS/lib/services/shipment.service.ts` wraps calls and maps rows to typed objects.
- **Inline fetch** — most dashboard/alert/milestone components call `fetch('http://localhost:5000/api/...')` in a `useEffect` (`AlertFeedTable`, `MilestoneBoard`, `ShipmentDetailModal`).

The logged-in user comes from `SAS/lib/hooks/useAuth.ts` (real session: email / role / department). Role screens pass that identity to the API so the server returns only what the user may see:

```
GET /api/shipments/all-milestones?role=operation&email=me@x.com
GET /api/alerts/active?role=super&department=SEA
```

3. Backend structure — where things live

`Backend/app.py` is the entry point: it creates the app, imports each route file as a *blueprint*, registers them, and starts the scheduler. Shared logic sits in `services/`.

Folder / file	Responsibility
<code>app.py</code>	Entry point: registers blueprints, starts APScheduler jobs.
<code>config.py</code>	Loads <code>Backend/.env</code> , exposes <code>SUPABASE_URL</code> / <code>SUPABASE_KEY</code> .
<code>routes/*.py</code>	One blueprint per area = the HTTP endpoints (the API).
<code>services/*.py</code>	Business logic + DB access, shared by routes and jobs.
<code>sync/*.py</code>	CargoWise -> Supabase synchronisation.
<code>migrations/*.sql</code>	Schema changes / test data (Supabase SQL editor).

4. Database connectivity

Credentials live in `Backend/.env` (untracked). `config.py` loads them:

```
# Backend/config.py
load_dotenv(os.path.join(BASE_DIR, '.env'))
SUPABASE_URL = os.getenv('SUPABASE_URL') or os.getenv('supabase_url')
```

```
SUPABASE_KEY = os.getenv('SUPABASE_KEY') or os.getenv('supabase_key')
```

The shared client is **lazy** — it opens a connection only on first use, so importing a module never crashes the app if env vars aren't ready. Every service/route imports this one object:

```
# Backend/services/supabase_client.py
supabase = LazySupabaseClient() # connects on first .table(...) call
supabase.reset() # drop a stale connection after sleep
from services.supabase_client import supabase
supabase.table('shipments').select('*').execute()
```

- **Backend** uses the **service key** (full access) for all reads/writes.
- **Frontend** uses the **anon key** (SAS/lib/supabase.ts) for a few direct reads.
- After the machine sleeps the pool can go stale (a 500 on the next call). `reset()` or a Flask restart fixes it.

5. The APIs and their files

Each block is a blueprint file and its endpoints. ★ marks the milestone / alerting / field-registry layer you built.

★ **Shipments** — Backend/routes/shipments.py

Endpoint	Method	What it does
/api/shipments	GET	All shipments (optional staff-code filter).
/api/shipments/stats	GET	Totals: total / delivered / delayed (role-scoped).
/api/shipments/stats/branch	GET	Delay breakdown per branch (Super dashboard).
/api/shipments/current-milestones	GET	Each shipment + its active milestone.
/api/shipments/all-milestones	GET	Every milestone per shipment (role-scoped).
/api/shipments/<id>	GET	One shipment + its milestones (detail modal).

★ **Alerts** — Backend/routes/alerts.py

Endpoint	Method	What it does
/api/alerts/active	GET	Overdue + delayed groups (role-scoped) - the Alert Feed.
/api/alerts/recompute	GET/POST	Run the status recompute on demand.
/api/alerts/<id>/status	PATCH	Update an alert's action status.

★ **Milestone library & templates** — Backend/routes/milestone_library.py / templates.py

Endpoint	Method	What it does
/api/milestone-library	GET/POST	List / create reusable milestone definitions.
/api/templates	GET/POST	List / create templates (ordered slots).
/api/templates/<id>/preview-assignment	GET	Which shipments a template matches.
/api/templates/<id>/assign	POST	Freeze a snapshot into shipment_milestones.

★ **Field Registry (naming mismatches)** — Backend/routes/field_map.py

Endpoint	Method	What it does
/api/field-map	GET/POST	List / add expected-field mappings.
/api/field-map/mismatches	GET	Registered fields missing from feed + look-alike.
/api/field-map/detect	POST	Detect mismatches and email the admin digest.

★ **Field Watch (delayed / renamed data)** — Backend/routes/field_watch.py

Endpoint	Method	What it does
/api/field-watch/scan	GET/POST	Recompute yellow field alerts; email new ones.
/api/field-watch/conflicts	GET	Mismatches deduped & grouped by template.
/api/field-watch/resolve	POST	Map once + clear all matching alerts.

★ **System settings** — Backend/routes/system_settings.py

Endpoint	Method	What it does
/api/system-settings/milestone-mismatch	GET/PUT	Recipient for mismatch emails.
/api/system-settings/field-watch	GET/PUT	Recipient for Field Watch emails.

Endpoint	Method	What it does
/api/system-settings/test-email	POST	Send a test email to the saved recipient.

Sync — Backend/routes/sync.py

Endpoint	Method	What it does
/api/sync	GET/POST	Trigger / read the CargoWise sync.
/api/sync/logs	GET	History of sync runs.
/api/sync/settings	GET/PUT	Sync schedule + alert prefs.

Other blueprints (not the milestone layer): auth, users, profile, change_password, security_settings, custom_tables, milestones, dashboard, notes, access_logs, audit_trail, alert_engine_routes, sales_digest_routes.

6. Syncing — CargoWise into the database

The sync pulls shipments from the CargoWise API and upserts them into Supabase, so the app works off a local copy. It runs on a schedule and can be triggered manually.

```
run_sync_job() # Backend/app.py
1. fetch_shipments_from_api() # services/cargowise_service.py
2. build_milestones(item) + load_field_map()
3. upsert_shipment(row) # services/supabase_service.py -> shipments
4. save_sync_log(...) / save_sync_error(...)
```

- **Fetch** — `services/cargowise_service.py` calls the CargoWise API.
- **Shape** — milestone fields fold into a milestones JSONB column; `milestone_field_map` (the Field Registry) says which API fields to collect.
- **Write** — `upsert_shipment()` writes to `shipments` (conflict on `cargowise_id`).
- **Milestone rows** — `sync/milestone_sync.py` keeps `shipment_milestones` in step; template assignment freezes a snapshot per shipment.

Field resolution: `services/field_registry.py` -> `resolve_field_value()` reads a milestone's value from the shipment's milestones JSONB, an alias, or a plain column — so renamed CargoWise fields still resolve once mapped.

7. Scheduled jobs (APScheduler, Asia/Colombo)

Started in `app.py`. They run only while Flask runs; a run missed during sleep is skipped.

Job	When	Function -> service
CargoWise sync	00:00 / 06:00 / 12:00 / 18:00 (+custom)	<code>run_sync_job</code> -> <code>cargowise</code> + <code>supabase_service</code>
Status recompute	every 30 min	<code>run_status_recompute</code> -> <code>status_recompute.py</code>
Field Watch scan	hourly :05 & :35	<code>run_field_watch</code> -> <code>field_watch.py</code>
Field mismatch email	hourly :15	<code>run_field_mismatch_check</code> -> <code>field_registry.py</code>
Alert engine	hourly :05	<code>run_alert_engine_job</code> -> <code>alert_engine.py</code>
Sales digest	daily 08:00	<code>run_sales_digest_job</code> -> <code>sales_digest.py</code>

8. Your layer — how the pieces fit

- **Status recompute** (`services/status_recompute.py`) — evaluates each milestone's frozen snapshot live and sets status: completed / overdue (deadline passed) / delayed (out of sequence) / pending.
- **Field Watch** (`services/field_watch.py`) — data integrity: an expected field is late and a renamed look-alike exists -> yellow card + email (separate recipient).
- **Field Registry** (`services/field_registry.py`, `routes/field_map.py`) — maps milestone fields to CargoWise fields; the admin page adds fields and resolves conflicts by template.
- **Role scoping** (`services/scope.py`) — `role+identity` -> allowed `shipment_ids`: sales -> `sales_user_email`, operation -> `milestone assigned_email`, super -> `transport_mode`.
- **Templates & library** — reusable milestone -> ordered template -> frozen snapshot in `shipment_milestones`.

Front-to-back trace (Current Milestones):

```
MilestoneBoard.tsx (scope='operation', email from useAuth)
-> GET /api/shipments/all-milestones?role=operation&email=...
-> routes/shipments.py: get_all_milestones()
-> services/scope.py: allowed_shipment_ids() (assigned_email)
-> supabase.table('shipments' / 'shipment_milestones')
<- { data: [ { shipment, milestones:[{status, due_date, field_alert}] } ] }
```

-> tabs Overdue / Delayed / Completed + By Client / By Member, colour-coded

9. Running it & environment

Piece	Command / file	Notes
Backend	python app.py (in Backend/)	Binds 127.0.0.1:5000. Needs Backend/.env.
Frontend	npm run dev (in SAS/)	Next.js on :3000.
Backend env	Backend/.env	SUPABASE_URL/KEY + SMTP_* (untracked).
Frontend env	SAS/.env.local	NEXT_PUBLIC_SUPABASE_URL/_ANON_KEY, NEXT_PUBLIC_API_

If an endpoint 404s right after a code change, or a query 500s after the laptop slept, it is almost always a stale Flask process — restart `python app.py` and confirm it binds :5000.